Simulation of Microstructure Evolution and Grain Growth Using Cellular Automata in MATLAB

- Members (1) - Felix Burkart

- Introduction

I am a material science engineering student. In my introductory materials class, Mat Sci 104, we learned about crystal structures of metals, and how processing metals affects the atomic properties of these structures. One particular materials processing method was annealing, where a worked metal was exposed to high temperatures over long periods of time. Through the annealing process, the crystal structure, initially composed of many small grains, would consolidate into one or many larger grains. This process would make the material more ductile, giving it different physical properties.

In my project, I am using MatLab code to simulate a 2D metal sample as it goes through the annealing process. Rather than explicitly modeling atomic diffusion or dislocation motion, this simulation uses a Monte Carlo approach where grain boundary motion emerges from probabilistic energy minimization

- Methods

My code runs a simulation based on a lattice Potts model commonly used to simulate grain growth. Broadly, the code workflow works like this:

- Execute function

- GUI Function
  - Run-Simulation Function
  - End
- End

Within the Run-Simulation Function, my code iterates for two for loops, the first for loop is the number of sweeps which is input by the user, and the second loop is running for every individual grain cell that exists. Within the latter, the loop picks a random cell, a random neighbor to that cell, and compares how the grain boundary energy of the surrounding cells would change. Through this system, the same cell can and likely is picked, compared, and changed more than once within one iteration of the larger loop, this is how randomness is taken into account in a Monte Carlo simulation.

Temperature plays an important role in this simulation. The higher the temperature is, the higher the likelihood that a grain switches orientation unfavorably. In-line the code and accompanying formula that dictate this are as follows:

```
if deltaE < 0 || rand() < exp(-deltaE/T)
    sample(row,col) = Cell_neighbor;
end
```

Figure 1 - Grain Swap Conditions with Metropolis Monte Carlo condition

Physically, this represents the idea that at higher temperatures, the probability of accepting energetically unfavorable orientation changes increases due to thermal fluctuations.This use of temperature is significant in accurately representing real grain growth, and is often the pushing force that encourages further grain growth. The grain boundary energy calculation follows a lattice Potts model, where neighboring cells with different orientations contribute to the system's total energy:

```
if Cell_neighbor ~= current_cell

    E_old = sum(neighbors ~= current_cell);
    E_new = sum(neighbors ~= Cell_neighbor);

    deltaE = E_new - E_old;
```

Figure 2 - Enthalpy comparison using Potts model assumption

The probability of accepting a change in orientation is then determined using the Metropolis Monte Carlo acceptance condition (described above).

- Results

First my code outputs a GUI, prompting the user to select the sample size, number of grain orientations, and temperature. These are important parameters within the simulation and will change not only what the overall output looks like but also how the simulation behaves (simulation behavior with temperature described above).

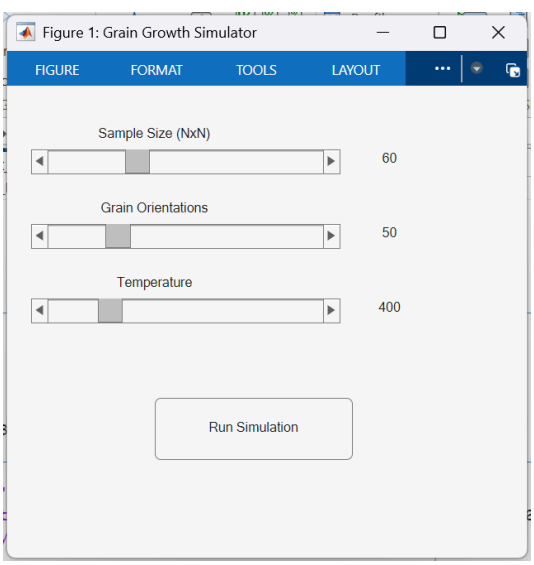

Figure 3 - Gui Prompting variable inputs

After selecting parameters, and pressing the "Run Simulation" button my code outputs the simulation and a graph and puts them side by side in a figure. On the left, the actual simulation of the grains is ran, this simulation updates at every loop iteration and displays grain

orientation both through a heatmap function and through the use of the quiver function, plotting arrows that point in consistent albeit arbitrary directions. On the right, a live graph is plotted, plotting the number of iterations (Monte Carlo Sweeps) against average grain area. Average grain area is calculated in-line using the following code:

```
num_grains = numel(unique(sample));
avg_area = (N*M)/num_grains;
avg_area_history(sweep) = avg_area;
```

Figure 4 - Code of Grain Size calculation

This takes the total size in units of the entire sample and puts it over the number of unique grain orientations, (quiver arrow directions).

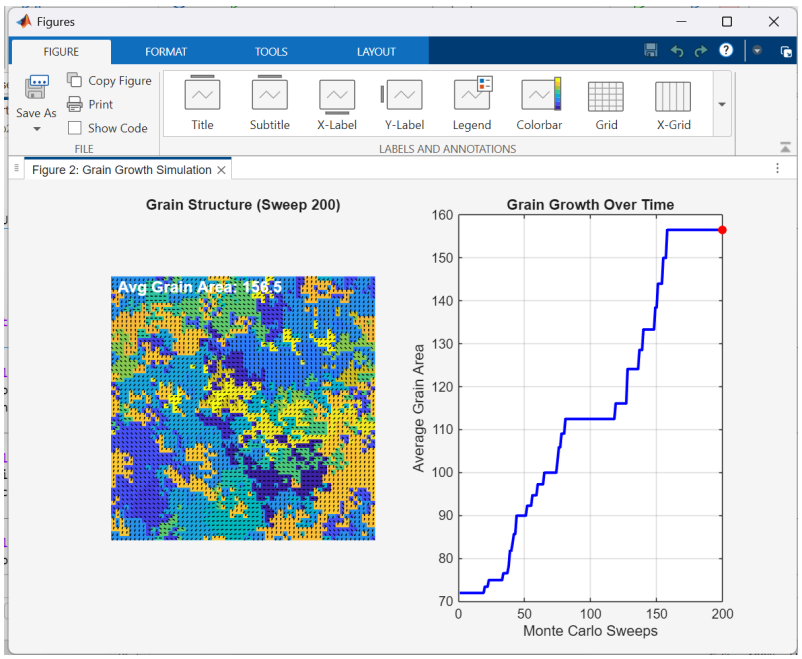

Figure 5 - Simulation output, live simulation and Monte Carlo Sweeps vs. Average Grain Area live plot

- Discussion

The code does a great job at fulfilling the requirements I set out for it. It effectively follows the Monte Carlo Simulation guidelines, and generates a realistic simulation that generally models an increase in average grain area as time (monte carlo sweeps) progresses.

I spent a long time trying to put in a checkbox to record the live simulation and graph once the simulation started. But unfortunately, I did not have the time to get it working. I kept on running into errors where the frames of the simulation would fail to be recorded, to overcome this I tried to include a failsafe where the error frame would just be skipped, but this resulted in every frame of the simulation being skipped and no video file being created.

Besides this major hurdle, I also struggled with aligning my conceptual understanding of annealing to what the code was modelling. I expected a clear linear relationship between

temperature and average grain area, but that was not at all the results I was getting. My model ended up being more accurate than my understanding of annealing. I think this misalignment came from the broad and absolute ways that I learned about annealing. Being such a dense class, Mat Sci 104 only really had time to say one thing about annealing:
higher temperature -> larger grain area.

- Conclusion

My biggest takeaways from this project are as follows:

1. Coding gets exponentially more complicated as you layer functionalities on top of each other
    a. It became increasingly important to separate sections of code according to intended goal
2. AI tools, while being incredible resources can become nonsensical code and statements quickly
    a. I went through a few rabbit holes of spiraling confusion
3. It is really important to take into account computational complexity when running simulations
    a. My computer got really hot before I set up limits to sample size, and simulation failsafes to skip loops at larger sizes

A future project expanding on this one would definitely include a method to record simulated data. It would also have included a better way of visualizing the grain areas, with boundary lines (I tried to do this but the product came out glitchy and inconsistent). Finally, the simulation would be more involved to model a more realistic grain growth environment, specifically one where the graph of Monte Carlo Sweeps vs. Average Grain Area doesn't exhibit sharp and sudden spikes but rather has a more gradual rise.

- Contributions

Felix Burkart - Everything

- References

Song, Xiaoyan, and Liu Guoquan. "A Simple and Efficient Three-Dimensional Monte Carlo Simulation of Grain Growth." *Scripta Materialia*, vol. 38, no. 11, 1998, pp. 1691-96, https://doi.org/10.1016/S1359-6462(98)00092-X.

Wright, Steven A., et al. *Potts-Model Grain Growth Simulations: Parallel Algorithms and Applications*. SAND97-1925, Sandia National Laboratories, Aug. 1997, doi:10.2172/522745.